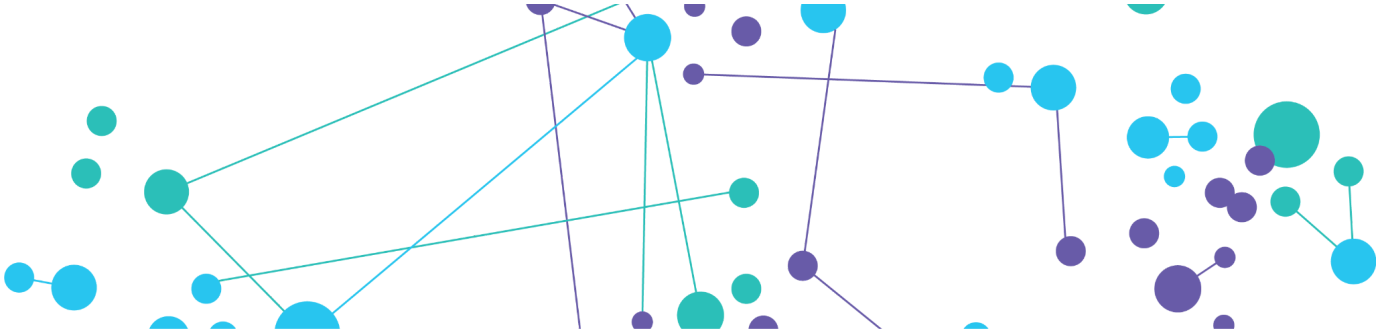
# Coding Project 2: JumboProve
# Coding Project 2: JumboProve

*Goals: Propositional Logic, Resolution, First Order Logic*

**Last Updated 10/6/26**

---



## Introduction

In this project, you'll work your way up to building a simplified resolution theorem prover for a reduced version of first-order logic.

First, you'll implement basic resolution propositional logic. Then, you'll build a generic "Resolution Prover", which performs a search to attempt to prove a statement. You'll then implement unification and resolution for first order logic. With those functions implemented, you can then use your Resolution Prover to prove first-order logic statements. For example, you'll be able to take knowledge bases like:

Grandparent(?x, ?y) ← Parent(?x, ?z) ∧ Parent(?z, ?y)
Parent(Brian, Doug)
Parent(Doug, Mary)
Parent(Mary, Sue)

And perform queries such as
Grandparent(?x, ?y)

Which will find all the grandparents in this knowledge base:
Grandparent(Brian, Mary)
Grandparent(Doug Sue)

While this is a simple example - if you can do this - your prover will be also able to solve more complex constraint satisfaction problems (such as map coloring)! Finally, you'll explore formalizing an existing problem, or how modern LLMs can generate predicates for existing problems.

Some notes
- Start early! This has a lot of steps to it, but I tried to break down each step into manageable chunks.
- Please look at the test files to see the expected behavior of each of the functions.

---

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

# Logistics

## Starter Code

Your HW is structured as follows:

```
coding_2/
├─ COLLABORATION.MD
├─ tests/
│  ├─ [tests...]
├─ knowledge_bases/
│  ├─ prop_kbs/
│  ├─ ├─ [kb].json
│  ├─ fol_kbs/
│  ├─ ├─ kb].json
├─ test.py
├─ FOLClauses.py
├─ PropClauses.py
├─ ResolutionProver.py
├─ sample_llm_prompt.md
├─ README.md
```

`COLLABORATION.MD:`

- An empty file where you will put your collaboration statement, listing anyone or anywhere you got help from. If you truly did not get help from anywhere or anyone, you must write that in your collaboration statement. **You will lose points if this is not completed.**

`tests/`:

- A set of tests for this assignment. There are 8 test files.

`knowledge_bases/`:

- A folder containing different knowledge bases in JSON format. The `kb` list consists of facts in the knowledge base in definite clause form, while the `goal` consists of the negated goal (see Set of Support resolution below).

`test.py`

- This file contains test cases for the functions you are going to implement in this assignment, much like prior assignments. **These will NOT be the only tests you are tested on**, and it is encouraged that you add your own test cases.

`PropClauses.py`

- Contains code that represents propositional clauses, as well as structure for a resolution search node for propositional logic.

`FOLClauses.py`

- Contains code that represents FOL clauses, as well as structure for a FOL search node for propositional logic.

`ResolutionProver.py`

- Contains code that can run a search algorithm on a knowledge base. In addition, has some code from loading knowledge bases in `.json` format.

`README.md`

- Some basic information on running the starter code.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

# Grade Breakdown

| Type | Feature | Points | Graded By |
|---|---|---|---|
| Part 1: Propositional Logic | Propositional Support Resolve | 10 (q1)<br>- 5 visible tests, 2 points each | auto |
| | Generate Propositional Successors | 10 (q2)<br>- 5 visible tests, 2 points each | auto |
| | Resolution Prover Pt 1: | 6 (q3)<br>- 6 visible tests, 1 point each | auto |
| Part 2: First Order Logic | Unification | 10 (q4)<br>- 10 visible tests, 1 points each | auto |
| | Chasing Bindings, Uniquify | 10 (q5)<br>- 10 visible tests, 1 each | auto |
| | FOL Support Resolve | 10 (q6)<br>- 10 visible tests, 1 point each | auto |
| | Generate FOL Successors | 10 (q7)<br>- 10 visible tests, 1 point each | auto |
| | Resolution Prover Test: Pt 2 | 14 (q8)<br>- 6 visible tests, 2 points each.<br>- 2 hidden tests, 1 point each | auto |
| | FOL Formalization or LLM Exploration | 10 | Manual |
| Meta | Style and Readability | 5 | Manual |
| | Collaboration Statement | 5 | Manual |
| **Total** | **Total** | 100 | N/A |

# Submission Instructions

In gradescope please upload the following files:

- `ResolutionProver.py`
- `FOLClauses.py`
- `PropClauses.py`
- `[Yourname]_Report.pdf (for the last question)`
- `For LLM-Free option:`
  - `Kb.json`
- `For LLM-Enabled Option:`
  - `Kb1.json`

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
      - kb2.json
  - COLLABORATION.MD
```

---

# Phase 1: Propositional Logic

## Set of Support Resolution

First, recall that a **definite clause** is of the form:

$$P \leftarrow P_1 \wedge P_2 \ldots \wedge P_n$$

where $P_1$, $P_2$...$P_n$ are all positive literals (i.e., positive atomic formulas). This should be read: if $P_1$, $P_2$...$P_n$ are all true, then P is true. Notice, that this formula has the equivalent clausal form:

$$P \vee \neg P_1 \vee \neg P_2 \ldots \vee \neg P_n$$

The general definition of a definite clause is that it is a disjunction with exactly one positive literal. This also means that a single positive literal by itself is also a definite clause. **All clauses in your knowledge base (both propositional and first order logic) will be definite clauses.** In the `kb` list of the `.json` files, the first literal of a clause is always positive, while the others are negative (this means that a clause with only one literal is positive).

Your **goal clause** will take the form of:

$$Q_1 \wedge Q_2 \ldots \wedge Q_m$$

where $Q_1$, $Q_2$...$Q_m$ are all positive literals. When we negate the goal clause (as we do when proving via resolution), this becomes:

$$\neg Q_1 \vee \neg Q_2 \ldots \vee \neg Q_m$$

As we discussed in class, restricting our clauses like this limits the expressivity of our logic. However, the benefit of this is that it leads to a relatively simple inference algorithm - which we'll call the **set of support strategy.** At a high level, the set of support represents all of the terms we'd need to prove in order to prove our goal.

Initialize our set of support as our negated goal clause. Notice that if we can successfully resolve our set of support with a clause in our knowledge base (which we limit to definite clauses), we will **always** get a clause that is still a disjunction of negative literals. Also note that since there's only one positive literal in our knowledge base clauses, if we can resolve our support set clause with a knowledge base clause, only one possible resolved clause can result. Take this resolved clause as our new set of support. We can keep repeating this process until we either reach the empty set (e.g proving our goal by finding a contradiction) or cannot resolve against anything in the knowledge base (meaning that we cannot prove our goal).

Programmatically, we can set this up by at each step trying to resolve the "leftmost" literals of the support set against the leftmost literal of all the clauses in our knowledge base.

### Example 1: Single Path

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

For example, consider the knowledge base:

1. A
2. B ← A
3. C ← A ∧ B

Which we can represent in CNF as:

1. A
2. B ∨ ¬A
3. C ∨ ¬A ∨ ¬B

1. Initialize our set of support *S* as our negated goal {¬C}.
2. We see that we can resolve against (3), creating a new set of support *S* = {¬A ∨ ¬B}
3. The leftmost literal of *S* is ¬A.
   We see that we can resolve it against (1), creating the new set of support *S* = {¬B}
4. The leftmost literal of *S* is ¬B.
   We see that we can resolve it against (2), creating the new set of support *S* = {¬A}
5. The leftmost literal of *S* is ¬A (again)
   We see that we can resolve it against (1), creating the new set of support *S* = {},
   which is the empty set, so thus our goal is proven.

For a visual, follow along the resolution steps in the **Simple Propositional Resolution** section here.

## Example 2: Branching Paths

Now, you may be wondering what happens if there are multiple clauses in our knowledge base that can be resolved with our support set? Say it with me folks: **we search!** There may be multiple paths to a goal, and we want to find all of them (especially in First Order Logic).

Let's say we add the rule *C* ← *A* ∧ *E* to our KB.

1. A
2. B ∨ ¬A
3. C ∨ ¬A ∨ ¬E
4. C ∨ ¬A ∨ ¬B

We wish to prove *C.*

1. Initialize our set of support *S* as our negated goal {¬C}.
2. We see that this clause can be resolved with both (3) and (4), so we branch. (We can keep track of this via a frontier stack or a queue, much like all of our searches).
3. **Branch 1 - Resolving against (3)**
   a. We see that we can resolve against (3), creating a new set of support *S* = {¬A ∨ ¬E}
   b. The leftmost literal of *S* is ¬A.
      We see that we can resolve it against (1), creating the new set of support *S* = {¬E}
   c. The leftmost literal of *S* is ¬E.
      This does not resolve against anything in our KB. Thus this branch **fails**.
4. **Branch 2 - Resolving against (4)**
   a. We see that we can resolve against (4), creating a new set of support *S* = {¬A ∨ ¬B}
   b. The leftmost literal of *S* is ¬A.
      We see that we can resolve it against (1), creating the new set of support *S* = {¬B}
   c. The leftmost literal of *S* is ¬B.
      We see that we can resolve it against (2), creating the new set of support *S* = {¬A}
   d. The leftmost literal of *S* is ¬A.
      We see that we can resolve it against (1), creating the new set of support *S* = {},

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

which is the empty set, so thus our goal is proven.

For a visual, follow along the resolution steps in the **Propositional Search Resolution** section here. This will be the basic algorithm for both the propositional and first order logic theorem prover. First, you'll implement the propositional version.

# Programming 1: Propositional Prover

## Problem 1.1: Propositional Support Resolve

In `PropClauses`, implement the function `support_resolve`. This function attempts to resolve the leftmost literal of a `support_clause` (which will always be a negative literal), with the leftmost literal of a `kb_clause` (which will always be a positive literal). If this resolution is successful, we return a `PropClause` which represents the result of the resolution (which given our constraints on support clause and knowledge base clauses, the returned clause will always consist of negative literals). If the two leftmost literals cannot be resolved, we return `None`.

Test your function with your `q1` tests.

## Problem 1.2: Generate Propositional Successors

Now, look in `ResolveNodeProp`. This is what we'll be using to run our search. The `self.support` variable is a `PropClause`, which we are interpreting as only consisting of negative literals.

In `ResolveNodeProp`, implement the `generate_successors` function. This function is given `kb`: a list of `PropClauses` representing our knowledge base. Using your `support_resolve` function, find the valid resolutions of the `self.support` clause against all the clauses in the `kb`. This function returns a list of `ResolveNodeProp`, representing these successor nodes.

Test this with your `q2` tests.

## Problem 2.3: Resolution Prover - Pt. 1

The `ResolutionProver` class takes in two variables: `kb` (our knowledge base) and `max_explored` (which will help us catch infinite loops when we work with First Order Logic). This function should be agnostic on whether it's proving a propositional statement or a first order logic statement.

`kb` can either be a list of `PropClause` or `FOLClause`. Both `PropClause` and `FOLClause` expose the methods `is_empty` and `generate_successors` (which you implemented for `PropClause` in the last problem). If you write it correctly for this problem, later in the assignment, you'll be able to swap in FOL clauses with no issue.

Now, in `ResolutionProver`, implement the `prove` function. Prove takes in a `goal`, which can be either a `ResolveNodeProp` or a `ResolveNodeFOL` (again, we only need to worry about `ResolveNodeProp` for now). This is quite similar to a simple graph search using a frontier, like you programmed in coding project 1. No need to create an explored set. In addition, there is no need to keep track of the "path" of the proof (e.g. the order of clauses you resolved against). However, you should find **all solutions** (not just exit when one empty set is found). While this isn't necessary for propositional proofs, this *is* required for first order proofs.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- `prove` returns
  - A list of the solution nodes, if there are solutions (either a list of `ResolveNodeProp` or of `ResolveNodeFOL`)
    - For `ResolveNodeProp`, this means that you'll be returning nodes with an empty support set. This is fine. When you go on to first-order logic, the `ResolveNodeFOL` nodes will contain the final bindings of possible solutions.
  - A boolean representing if the search terminated or not, which is false if we exceeded the maximum amount of nodes explored.

Test this with your `q3` tests. You can also load in `.json` knowledge bases using the `load_prop_kb` function.

# Programming 2.1: First Order Logic

The algorithm for proving first-order logic statements is exactly the same as propositional logic - but requires implementation of unification and keeping track of potential bindings through our resolutions. Let's go through an example.

Consider the KB:

1. Grandparent(?x, ?y) ← Parent(?x, ?z) ∧ Parent(?z, ?y)
2. Parent(Brian, Doug)
3. Parent(Doug, Mary)
4. Parent(Mary, Sue)

Converting to CNF

1. Grandparent(?x, ?y) ∨ ¬Parent(?x, ?z) ∨ ¬Parent(?z, ?y)
2. Parent(Brian, Doug)
3. Parent(Doug, Mary)
4. Parent(Mary, Sue)

We want to prove *Grandparent(?x, ?y)*. Who are all the grandparents in our KB?

This is best shown with a visual, so follow along the resolution steps in the **FOL Resolution** section here. I won't clog up this document with more text, but if you want, I wrote it out **here** as well. But you'll see that our algorithm returns with the two valid resolutions:

- {?x / Brian, ?y / Mary}
- {?x / Doug, ?y / Sue}

Some Notes:

- All variables will be prefaced with a question mark, for example: `?x`, `?y`, `?dog`
- We'll represent bindings with python dictionaries, for example:
  - `{"?x": "?y", "?y":"JOE"}`
  - This reads as `?x` is bound to variable `?y`, and `?y` is bound to constant `JOE`.

## Problem 2.1: Unify

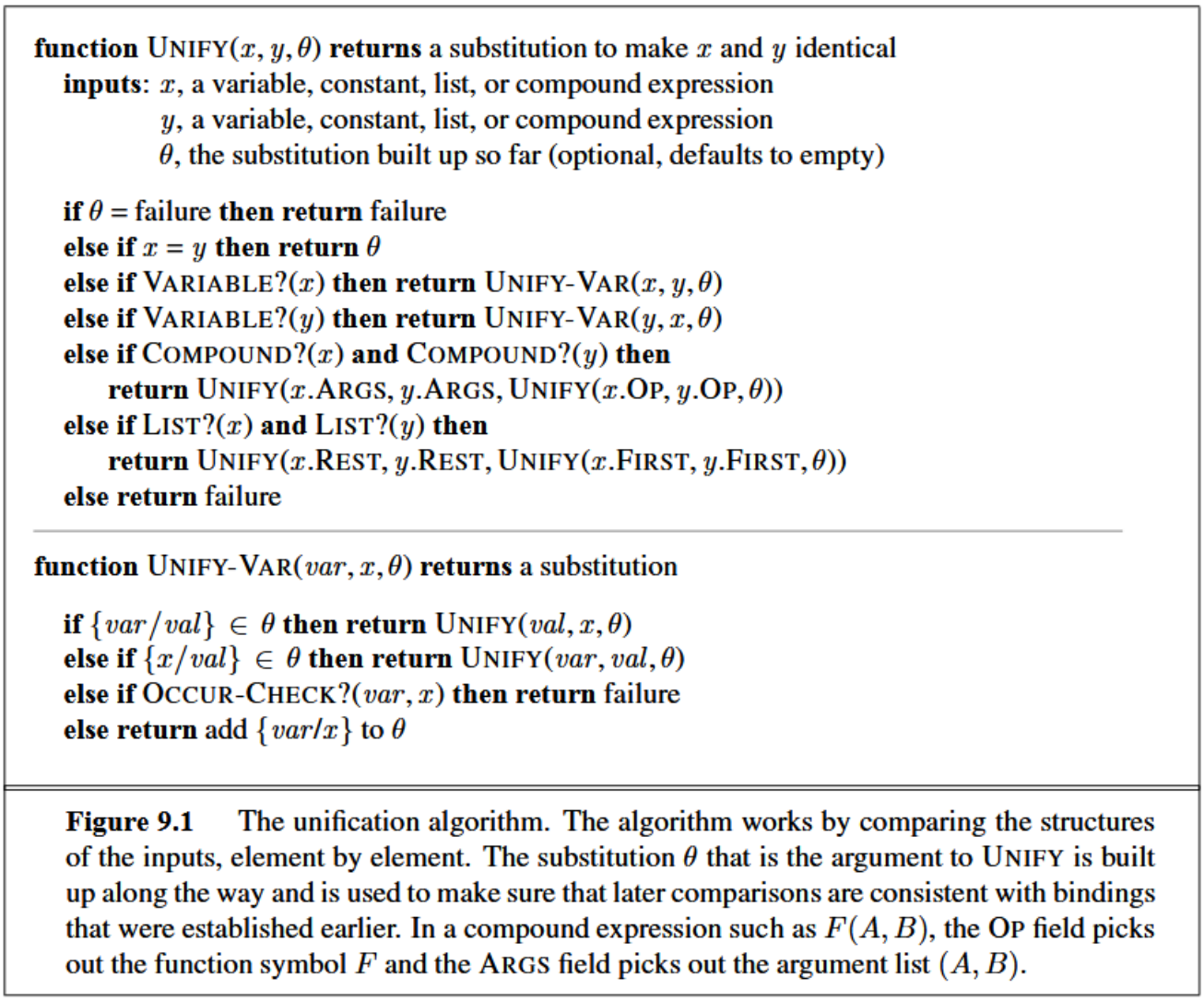
In `FOLAtom` implement the `unify` function. For this, I would highly recommend following the provided pseudocode in R&N chapter 9 (duplicated below.)

Some notes:

- Since we are not including functions in our prover, you can ignore the `Compound?` check.
- You can similarly ignore the `Occur-Check?`, as we won't be doing self referential statements.
- The bindings you return will be a dictionary, as demonstrated above.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- You should return `None` if no bindings can be found (e.g. `failure` in the provided pseudocode below).



**function** UNIFY($x, y, \theta$) **returns** a substitution to make $x$ and $y$ identical
  **inputs**: $x$, a variable, constant, list, or compound expression
    $y$, a variable, constant, list, or compound expression
    $\theta$, the substitution built up so far (optional, defaults to empty)

  **if** $\theta$ = failure **then return** failure
  **else if** $x = y$ **then return** $\theta$
  **else if** VARIABLE?($x$) **then return** UNIFY-VAR($x, y, \theta$)
  **else if** VARIABLE?($y$) **then return** UNIFY-VAR($y, x, \theta$)
  **else if** COMPOUND?($x$) **and** COMPOUND?($y$) **then**
    **return** UNIFY($x$.ARGS, $y$.ARGS, UNIFY($x$.OP, $y$.OP, $\theta$))
  **else if** LIST?($x$) **and** LIST?($y$) **then**
    **return** UNIFY($x$.REST, $y$.REST, UNIFY($x$.FIRST, $y$.FIRST, $\theta$))
  **else return** failure

**function** UNIFY-VAR($var, x, \theta$) **returns** a substitution

  **if** $\{var/val\} \in \theta$ **then return** UNIFY($val, x, \theta$)
  **else if** $\{x/val\} \in \theta$ **then return** UNIFY($var, val, \theta$)
  **else if** OCCUR-CHECK?($var, x$) **then return** failure
  **else return** add $\{var/x\}$ to $\theta$

**Figure 9.1** The unification algorithm. The algorithm works by comparing the structures of the inputs, element by element. The substitution $\theta$ that is the argument to UNIFY is built up along the way and is used to make sure that later comparisons are consistent with bindings that were established earlier. In a compound expression such as $F(A, B)$, the OP field picks out the function symbol $F$ and the ARGS field picks out the argument list $(A, B)$.

Test it with `q4` tests.

## Problem 2.2: Chasing Bindings, Uniquify

Eventually, when we print our solution bindings, we often don't care about the intermediate bindings of our program.

First, in `FOLAtom`, implement `get_terminal_bindings`. This takes in a variable `var` and dictionary of `bindings` and "chases" `var` through the bindings transitively until it can't anymore - returning the final binding. For example:

- If we have `bindings = {"?x": "?y","?z":"BOB", "?y":"JOE"}`
- `get_terminal_bindings("?z", bindings)` will return `"BOB"`.
- `get_terminal_bindings("?x", bindings)` will return `"JOE"`.
- `get_terminal_bindings("?y", bindings)` will return `"JOE"`.

In `FOLClause` implement `get_final_bindings`. Given a list of `intermediate_bindings`, this returns the terminal bindings for all the variables present in the clause and does not show intermediate bindings. For example:

- If we have a clause of the form:
  - `clause = [ ["PARENT", "?x", "?z"], ["PARENT", "?z", "?y"]]`

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

    - And `bindings = {“?x”: “JOE”, “?z”: “?w”, “?y”: “ALICE”, “w?”:”BOB”}`
    - `clause.get_final_bindings(bindings)` returns:
        - `{“?x”: “JOE”, “?z”: “BOB”, “?y”: “ALICE”}`

Finally, implement the `uniquify` function, which returns an `FOLClause` with the variables renamed uniquely. You will need to use this function before each unification to guarantee a “fresh” instance of the knowledge base clause you are unifying your support set against. This is because you may be resolving a premise clause with another clause derived from that same premise, and the variables from the two instances must not be the same. There are multiple ways you could go about this, either by keeping track using a global counter variable or using uuids.

For example:
- `clause = [ ["PARENT", "?x", "?z"], ["PARENT", "?z", "?y"] ]`
- `FOL.uniquify(clause)` could result in something like
    - `[ ["PARENT", "?x-89", "?z-6"], ["PARENT", "?z-6", "?y-42"] ]`

You can test your `get_terminal_bindings` and `get_final_bindings` with `test`.
We can’t test your `uniquify` function because it is unique to how you implement it.

Test your function with `q5` tests.

## Problem 2.3: FOL Support Resolve

In `FOLClause`, implement the function `support_resolve`. Much like the propositional version of this function, this function attempts to resolve the leftmost atom of a `support_clause` (which will always be a negative), with the leftmost atom of a `kb_clause` (which will always be a positive). The only difference is now we have to uniquify the `kb_clause` and see if the leftmost atoms of the uniquified `kb_clause` and the `support_clause` can be unified.

If the two atoms can be unified, we can resolve the two clauses.

We then return *two* things:
- An `FOLClause` which represents the result of the resolution (which given our constraints on support clause and knowledge base clauses, the returned clause will always consist of negative atoms).
- A `bindings` dictionary, representing the result of the unification.

If the two leftmost literals cannot be resolved, we return `None`.

Test your function with `support_resolve` tests. These will only check if you get the right number of atoms, the predicates line up, and if you have the correct number of bindings. We can’t check if the bindings are correct due to the uniquification constraint.

Test your function with `q6` tests.

## Problem 2.4: Generate FOL Successors

Now, look in `ResolveNodeFOL`. This will be very similar to how you implemented the propositional generate successors function. The `self.support` variable is a `FOLClause`, which we are interpreting as only consisting of negative literals. The only difference is now each node also has to keep track of the current bindings, represented by `self.bindings`.

In `ResolveNodeFOL`, implement the `generate_successors` function. This function is given `kb`: a list of `FOLClauses` representing our knowledge base. Using your FOL `support_resolve` function, find the valid resolutions of the `self.support` clause against all the clauses in the `kb`. This function returns a list of `ResolveNodeFOL`, representing these successor nodes. Make sure to keep track of the bindings

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Test your function with `q7` tests.

## Problem 2.5: Resolution Prover - Pt. 2

Now, if you set up your `prove` correctly, you should be able to swap in a `ResolveNodeFOL` with no issue. For example, if you load in the knowledge base `load_fol_kb` with `"family_tree.json"`, it models our original grandparent problem. This should return two solution nodes, and you'll see that if you print out the bindings (after chasing through everything to their terminal bindings), you'll get the two solution bindings:

*{'?x': 'BRIAN', '?y': 'MARY'}*
*{'?x': 'DOUG', '?y': 'SUE'}*

- `prove` returns
  - A list of the solution nodes, if there are solutions (either a list of `ResolveNodeProp` or of `ResolveNodeFOL`)
    - When you go on to first-order logic, the `ResolveNodeFOL` nodes will contain the final bindings of possible solutions.
  - A boolean representing if the search terminated or not, which is false if we exceeded the maximum amount of nodes explored.

Try this out with all the other knowledge bases provided. Remember, you need to find **all solutions** (and not just exit when the empty set is found). The knowledge bases provided:

- `family_tree.json`: Our standard grandparent problem.
- `family_2.json`: Related to grandparents, showcases aunts..
- `Ancestors.json`: Showcases how you can handle recursive inference.
- `Ancestors_infinite.json`: But showcases that you have to be careful about ordering, lest you can get put in an infinite loop (can you see why?).
- `down_under.json` : Our Australia map problem. See how we represent the diff constraints?
- `Sentences.json`: A more complicated natural language parsing generator. Gives a lot of valid bindings.

Test your function with `q8` tests.

## Problem 2.6: Formalizing Your Own Problem

Congrats, you've now built a first order logic theorem prover! Now, you'll use it to solve a puzzle of your own choosing.

Look at the wikipedia page for *Logic Puzzles* and pick one that seems interesting - then, attempt to formalize it in the first order logic form we provided. Look at the knowledge bases in `fol_kbs` for inspiration.

I'm giving you one of two options on how to complete this last part of the assignment, either **LLM-Free** or **LLM-Enabled.** Both are worth exactly the same amount of points. **Choose one**.

### LLM-Free

Choose a logic puzzle and formalize it in the `.json` format, such that your theorem prover can find valid solutions.

Some suggestions:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- I would stick to only the simplest of problems, and their smallest possible cases - due to the limitation of our language, you may have to specify a lot of “obvious” or redundant constraints (e.g. for example, see the `down_under.json` file for how we design the Australia map coloring problem.)
    - This can get quickly cumbersome for `Alldiff` style constraints (as you’d have to specify `(Alldiff 1 2 3)`, `(Alldiff 1 3 2)`... etc.. for every permutation! However, if you want, you could also write a program that *generates* these base constraints as a sort of preprocessor…)

**If you’re doing the LLM-Free option, your submission must contain:**

- A small pdf file of a report, which includes:
    - A description of the problem you chose.
    - A one to two paragraph description of your process of formulating the problem. You must include how you verified your formulation is correct - such as a comparison against known solutions for small instances of the problem.
        - You must clearly explain each rule in your knowledge base. You must clearly explain each clause in your knowledge base and why it is needed.
- A `.json` file of your problem representation and the goal. We should be able to load your `.json` into the theorem prover using the `load_fol_kb` function we provided and run our resolution prover.

## LLM-Enabled

If you choose this option, you’ll attempt to have a modern LLM provider generate a valid first order logic representation of a logic puzzle such that it can be run by your prover.

Choose a logic puzzle and ask **at least two different LLMs** to generate a representation in the `.json` format, such that your theorem prover can find valid solutions.

Some notes:

- For this assignment, I am explicitly allowing you to upload the first order logic `.json` files to any third party LLM you wish (e.g. ChatGPT, Claude, etc). **These are the only files you are allowed to upload.**
- You must try **at least two different models from different providers**.
- Give the LLM an example of the specification and limitations of your prover, as well as a specification of the puzzle you want to generate the constraints of.
    - I’ve given a sample prompt in `sample_llm_prompt.md`
- Note: Depending on which LLM and puzzle you choose, it may advise that due to the limited representational power of our logic, that it will generate a cumbersome response. Let it - see what happens.

**If you’re doing the LLM-Enabled option, your submission must contain:**

- A small pdf file of a report, which includes:
    - A description of the problem you chose.
    - Two to three paragraph description of your process of formulating the problem and interactions with the LLMs. How did the LLM formulate the problem? What worked? What didn’t? Did it do anything interesting? How did the two models differ? Does manipulating the prompt give different results?
        - You must clearly explain each rule in your knowledge base **in your own words** and why it is needed.
    - You must include how you verified the formulation is correct - such as a comparison against known solutions for small instances of the problem.
    - In addition, your report **must include**
        - **Full transcripts of your interactions with the models.**
        - The *specific* model(s) you used
            - (e.g. not just “ChatGPT and Claude”, state “OpenAI GPT-5, Anthropic Claude-Opus-4.6)
        - The date which you accessed these models and generated the content

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Two `.json` files of the different LLM problem representations and the goal. We should be able to load the `.json` into the theorem prover using the `load_fol_kb` function we provided and run the resolution prover and get a result.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF